Multi-Agent Deep RL for Decentralized Traffic-Signal Coordination: Project Report

Nov 7, 2026 · @Ayush

Abstract

This project trains deep Q-network (DQN) agents to control traffic signals at four intersections and compares two ways of sharing learning across them: independent agents (one network each) and a shared-parameter agent (one network used by all four). Both are evaluated against Random, Fixed-Time and Max-Pressure controllers in a lightweight queue-based simulator. The codebase is complete and runnable; benchmark numbers are not yet reported here and must come from your own training runs (see Evaluation).

Problem formulation

Each intersection is one agent in a decentralized partially observable setting (Dec-POMDP): it sees only local and neighbouring queue information and picks its own signal phase every 5 seconds.

- **Observation (17 values):** 4 incoming queue lengths, 4 mean lane speeds, a one-hot of the current phase (4), phase dwell time in minutes, and 4 copies of the mean downstream queue.
- **Action (4 choices):** which of the 4 signal phases to run next.
- **Action masking:** until a phase has run for the 10 s minimum green, only "keep current phase" is allowed. The mask is applied both when exploring and when picking the greedy action.
- **Reward:** the negative of local queue length plus 0.1 times local delay, minus 0.2 times downstream queue length.

```
r_i = -\left(\sum_{l} q_l + 0.1 \sum_{l} d_l\right) - 0.2 \sum_{k}
q_k^{\text{down}}
```

Here q is queue length, d is delay (1.5 s per queued vehicle), and the last sum runs over the four outgoing lanes.

Simulation environment

The environment (`src/env.py`) is a simplified queue model of four independent intersections, each with 4 incoming and 4 outgoing lanes. It is not SUMO and does not simulate individual vehicles.

- **Time step:** one decision every 5 s; an episode lasts 3,600 s (720 decisions).
- **Arrivals:** each incoming lane receives Poisson(0.8) vehicles per decision.
- **Discharge:** phase 0 serves lanes 0 and 2, phase 2 serves lanes 1 and 3, releasing 2 or 3 vehicles per step. Phases 1 and 3 discharge nothing.
- **Lane state:** delay is 1.5 s per queued vehicle; speed is 13.89 m/s on an empty lane and 2.5 m/s otherwise.
- **Safety wrapper:** enforces the 10 s minimum green through the action mask. The 3 s yellow time is stored as a parameter but is not simulated.
- **Reset:** queues start at 0 to 4 vehicles per lane, downstream queues at 0 to 2, seeded for reproducibility.

Algorithms

Both learners use the same DQN (`src/agent.py`): a Q-network of 17 → 128 → 128 → 64 → 4 units with ReLU, a target network copied every 500 updates, an experience replay buffer of 50,000 transitions, and the Bellman target below.

```
y = r + (1 - \text{done}) \, \gamma \max_{a'} Q_{\text{target}}(s', a')
```

The two variants differ only in how networks and replay buffers are shared.

Variant	Networks	Replay buffer	Intended benefit
Independent DQN	One per intersection	One per intersection	Each agent specializes to its own traffic
Shared-parameter DQN	One for all four	One shared	Four times the experience per update

Default hyperparameters: learning rate 1e-3 (Adam), discount 0.95, batch size 64, MSE loss. Exploration starts at ε = 1.0 and decays linearly to 0.05 over the first 80% of episodes. Default training is 50 episodes per seed.

Baselines

Three non-learning controllers are implemented in `src/baselines.py` as reference points.

Controller	Rule	Uses queue information
Random	Picks uniformly among the phases allowed by the mask	No
Fixed-Time	Cycles through phases, switching every 30 s	No
Max-Pressure	Picks the phase with the largest incoming-minus-outgoing queue difference	Yes, including downstream

The controllers are written but not yet wired into an evaluation script, so no baseline results exist yet. Adding that script is the main missing piece for a full comparison.

Implementation and reproduction

The project is plain Python with PyTorch, NumPy, pandas and matplotlib.

File	Purpose
`src/agent.py`	DQN agent, Q-network, replay buffer, action-masked selection
`src/env.py`	Four-intersection simulator and safety mask
`src/baselines.py`	Random, Fixed-Time and Max-Pressure controllers
`src/train.py`	Training loop for independent or shared DQN; writes per-seed CSV logs
`src/plot_curves.py`	Mean ± standard deviation learning curves across seeds

To reproduce, install the requirements and train both variants on several seeds, then plot.

```
pip install -r requirements.txt
cd src
for seed in 1 2 3; do
  python train.py --algo shared_dqn      --seed $seed --episodes 50
  python train.py --algo independent_dqn --seed $seed --episodes 50
done
python plot_curves.py --results ../results
```

Each run writes `results/<algo>_seed<seed>.csv` with return, loss, waiting time and queue length per episode, and the plot is saved as `results/learning_curves.png`.

Evaluation and results

No results are reported yet: the project has not been trained to completion, and any table of numbers here would have to come from your runs. Report only measured values.

A sound evaluation would follow these steps.

1. Train both DQN variants on at least 3 seeds and keep the CSV logs.
2. Run Random, Fixed-Time and Max-Pressure in the same environment, with the same episode length and seeds.
3. Evaluate trained agents with exploration off ($\varepsilon = 0$) on fresh seeds, not the training episodes.
4. Report mean ± standard deviation of average waiting time, average queue length and cumulative return.
5. Add a short results table and the learning-curve figure to this section.

Limitations and future work

The simulator is a simplified model, so conclusions about real traffic would not carry over directly.

- **Intersections are not coupled.** Downstream queues are set at reset and never change, so one agent's decisions do not affect its neighbours. This weakens any claim about coordination.
- **Phases 1 and 3 discharge no vehicles,** so the agent can only learn to use phases 0 and 2 productively.
- **Yellow time is not simulated,** and the discharge rule is a rough approximation of saturation flow.

- **Arrivals are stationary.** There is no peak-hour or tidal demand, so robustness to changing traffic is untested.

Natural next steps:

1. Link neighbouring intersections so outgoing vehicles become the next intersection's arrivals.
2. Add an evaluation script for the baselines and trained agents.
3. Add time-varying demand to test peak-hour behaviour.
4. Port the agents to SUMO for realistic vehicle dynamics.